

Local DSC Helper Service — Phase 1

Cloud-to-Local USB DSC Integration Engine

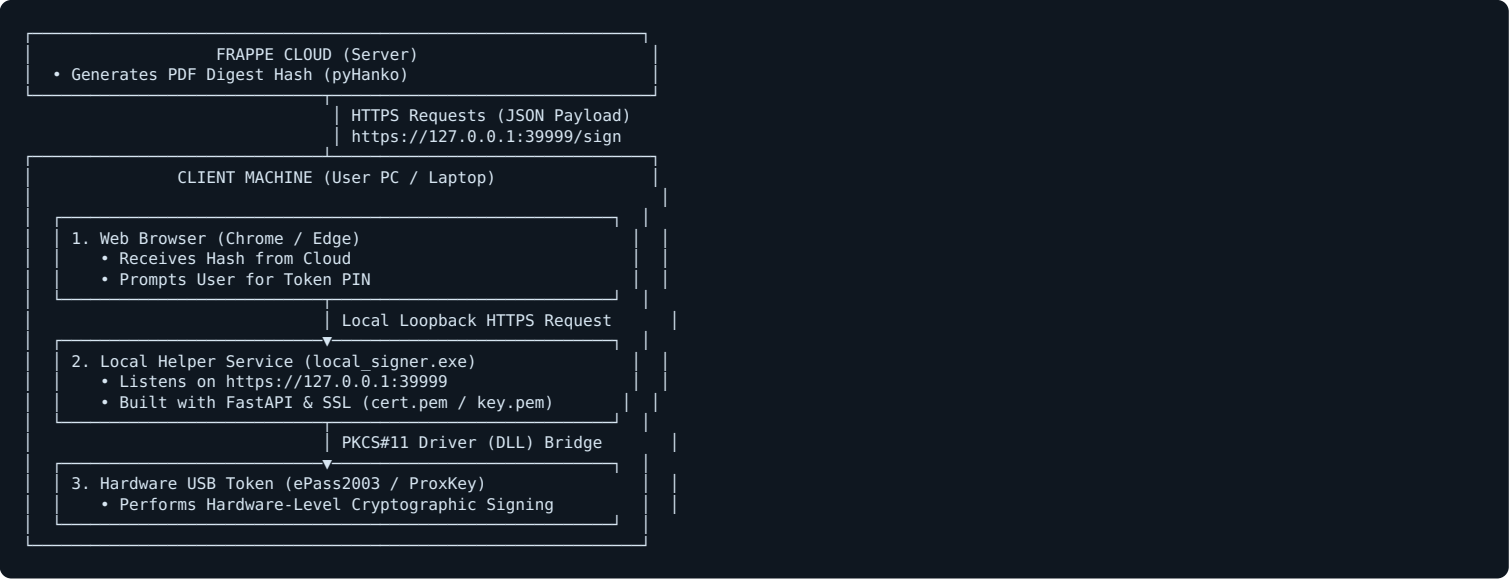
Target Environment	Windows Client OS (Windows 10 / 11)
Core Stack	Python 3.x, FastAPI, Uvicorn, python-pkcs11, cryptography, PyInstaller
Component	local_signer.exe (Local Loopback Signing Service)
Protocol	HTTPS Loopback (127.0.0.1:39999) + PKCS#11
Document Type	Implementation Guide / Developer Runbook

Table of Contents

- 1. System Architecture Blueprint
- 2. Developer Step-by-Step Implementation Guide
 - Step 1 — Directory Creation & Environment Isolation
 - Step 2 — Dependencies Installation
 - Step 3 — Local SSL Certificate Generation
 - Step 4 — Core FastAPI Service Application
 - Step 5 — Service Execution & Local Browser Verification
 - Step 6 — Single Executable Distribution Build (.exe)
- 3. Client Distribution Instructions Summary

1. System Architecture Blueprint

Yeh document Frappe Cloud se lekar client ki local USB DSC (Digital Signature Certificate) token tak, poore signing flow ka end-to-end architecture aur implementation describe karta hai. Neeche diya gaya diagram teeno layers — Cloud, Browser, aur Local Helper Service — ke beech data flow dikhata hai.



Flow Summary

Frappe Cloud pehle PDF ka digest hash generate karta hai (pyHanko ke through), phir woh hash browser ke through local loopback HTTPS endpoint par bheja jaata hai. Local helper service token PIN validate karke, PKCS#11 driver ke zariye hardware token par hash ko sign karta hai, aur signature wapas cloud ko return kar deta hai.

Layer	Component	Responsibility
Server	Frappe Cloud	PDF digest hash generation (pyHanko)
Client — Browser	Chrome / Edge	Hash receive karta hai, user se PIN maangta hai
Client — Local Service	local_signer.exe (FastAPI + SSL)	Loopback HTTPS listener, PKCS#11 bridge
Client — Hardware	USB Token (ePass2003 / ProxKey)	Hardware-level cryptographic signing

2. Developer Step-by-Step Implementation Guide

Is section mein har step ka executed command, uska purpose (Kyu Kiya?), aur expected outcome diya gaya hai.

1 Directory Creation & Environment Isolation

EXECUTED COMMANDS

```
cd C:\
mkdir DSC_Local_Signer
cd DSC_Local_Signer
python -m venv venv
venv\Scripts\activate
```

Kyu Kiya? (Why We Did This)

- `mkdir DSC_Local_Signer` — Root directory (C:\) mein project-specific space banaya gaya taaki compiled .exe dependencies system drive ke fixed path par accessible rahein.
- `python -m venv venv` — Ek isolated Virtual Environment banaya gaya, taaki system ki global Python libraries ke saath version conflicts na ho.
- `venv\Scripts\activate` — Environment activate kiya gaya taaki aage install hone waali sabhi libraries sirf is project folder mein locked rahein.

2 Dependencies Installation

EXECUTED COMMAND

```
pip install fastapi uvicorn python-pkcs11 cryptography
```

Libraries & Purpose

Library	Purpose
fastapi	Light-weight, high-performance web framework — local loopback endpoint (/sign) expose karta hai
uvicorn	ASGI server interface jo FastAPI backend ko execute karta hai
python-pkcs11	PKCS#11 standard ka Python wrapper — Hardware USB Tokens ke driver (.dll) se direct talk karta hai
cryptography	Local SSL certificate (cert.pem & key.pem) generate karne ke liye cryptographic primitives

3 Local SSL Certificate Generation (generate_certs.py)

Context

Frappe Cloud `https://` par run hota hai. Security guidelines ke mutabiq, `https://` site kisi non-SSL `http://127.0.0.1` API ko Mixed Content Block kar deti hai. Isiliye local API ko SSL-enabled karna compulsory hai.

PYTHON SCRIPT — GENERATE_CERTS.PY

```
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization
import datetime

key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
subject = issuer = x509.Name([x509.NameAttribute(NameOID.COMMON_NAME, u"127.0.0.1")])

cert = x509.CertificateBuilder().subject_name(subject).issuer_name(issuer)\
    .public_key(key.public_key())\
    .serial_number(x509.random_serial_number())\
    .not_valid_before(datetime.datetime.utcnow())\
    .not_valid_after(datetime.datetime.utcnow() + datetime.timedelta(days=3650))\
    .sign(key, hashes.SHA256())

with open("key.pem", "wb") as f:
    f.write(key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.TraditionalOpenSSL,
        encryption_algorithm=serialization.NoEncryption()))

with open("cert.pem", "wb") as f:
    f.write(cert.public_bytes(serialization.Encoding.PEM))

print("SSL Certificates (cert.pem & key.pem) Successfully Created!")
```

EXECUTION COMMAND

```
python generate_certs.py
```

Outcome

`C:\DSC_Local_Signer` folder mein `cert.pem` aur `key.pem` generate hue.

4 Core FastAPI Service Application (local_signer.py)

Is file ke andar main business logic aur USB Token Interfacing hai.

APPLICATION CODE — LOCAL_SIGNER.PY

```
import os
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import pkcs11
from pkcs11 import ObjectClass, Attribute

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

class SignPayload(BaseModel):
    hash: str
    pin: str

@app.post("/sign")
def sign_hash_payload(payload: SignPayload):
    possible_dlls = [
        r"C:\Windows\System32\eps2003csp11v2.dll",
        r"C:\Windows\System32\ep2000pk11.dll",
        r"C:\Windows\System32\eps2003csp11.dll"
    ]

    dll_path = next((p for p in possible_dlls if os.path.exists(p)), None)
    if not dll_path:
        raise HTTPException(status_code=500, detail="USB Token Driver (DLL) system mein nahi mila!")

    try:
        lib = pkcs11.lib(dll_path)
        slots = lib.get_slots(token_present=True)
        if not slots:
            raise HTTPException(status_code=400, detail="USB Token PC se connected nahi hai!")

        token = slots[0].get_token()

        with token.open(user_pin=payload.pin) as session:
            priv_keys = list(session.get_objects({ObjectClass.PRIVATE_KEY: True}))
            if not priv_keys:
                raise HTTPException(status_code=404, detail="Token mein Private Key nahi mili!")

            key = priv_keys[0]
            raw_hash = bytes.fromhex(payload.hash)

            # Hardware Level Signing
            signature = key.sign(raw_hash)
            return {"status": "success", "signature": signature.hex()}

    except pkcs11.exceptions.PinIncorrect:
        raise HTTPException(status_code=401, detail="Galat PIN enter kiya gaya hai!")
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=39999, ssl_keyfile="key.pem", ssl_certfile="cert.pem")
```

How Code Works Under the Hood

- **CORS Middleware** — Browser Cross-Origin Security violation avoid karta hai.
- **DLL Auto-Detection** — Standard Indian DSC Tokens (ePass2003 / HYP2003) ke drivers ko System32 path par inspect karta hai.
- **Hardware Session** — Token ko User PIN se authenticate karta hai aur Token Hardware Cryptographic Processor ka use karke PDF Hash Digest ko sign karta hai.

5 Service Execution & Local Browser Verification

EXECUTION COMMAND

```
python local_signer.py
```

Verification Step	Detail
User opened	https://127.0.0.1:39999/docs
Expected Warning	NET::ERR_CERT_AUTHORITY_INVALID (Self-Signed Cert ki wajah se normal warning)
Action Taken	Clicked Advanced → Proceed to 127.0.0.1 (unsafe)
Result	FastAPI Swagger Interactive Interface successfully load hua

6 Single Executable Distribution Build (.exe)

End-users/clients ke PC par Python environment install na karna pade, isliye poori application ko single Windows binary container mein compile kiya gaya.

EXECUTED COMMANDS

```
pip install pyinstaller
pyinstaller --noconfirm --onedir --windowed --add-data "cert.pem;." --add-data "key.pem;." local_signer.py
```

Command Breakdown (Kyu Kya Parameter Diya?)

Parameter	Purpose
--noconfirm	Purane build caches ko bina prompt ke overwrite karta hai
--onedir	Saari binary dependencies aur DLLs ko ek self-contained folder mein organize karta hai
--windowed	Background Service Mode enable karta hai (koi annoying CMD window open nahi hogi)
--add-data "cert.pem;." --add-data "key.pem;."	Self-signed SSL keys ko final .exe package ke root scope mein embed karta hai

Output Location

C:\DSC_Local_Signer\dist\local_signer folder mein standalone local_signer.exe ready ho chuki hai.

3. Client Distribution Instructions Summary

Client system par is service ko setup karne ke liye neech diye gaye steps follow karein:

#	Instruction
1	Client Machine par unka Official USB Token Driver (ePass2003 / ProxKey) Installed hona chahiye.
2	C:\DSC_Local_Signer\dist\local_signer folder ko Client PC par copy karein.
3	local_signer.exe ko Run Karein (ya Windows Startup folder shell:startup mein paste kar dein taaki har boot par auto-start ho jaaye).

Operational Notes

- Self-signed certificate hone ki wajah se browser pehli baar security warning dega — user ko manually "Proceed to 127.0.0.1 (unsafe)" accept karna hoga.
- CORS policy abhi allow_origins=["*"] par set hai; production rollout se pehle isse specific trusted origin(s) tak restrict karna recommended hai.
- Token PIN kabhi bhi disk par store nahi hoti — sirf request payload ke through runtime par use hoti hai.